Independent University, Bangladesh

CSE-313 Mid + Quiz Exam, Autumn-23, SET-A · Time: 1h30m

Real paper from this course. Quiz questions are on the other side of the paper.

Question paper

Mid Question (Total marks: 20)

1. Prove whether the following Grammar is Ambiguous or NOT. If the following grammar is ambiguous, convert it to unambiguous form. **[7]**

```
E → E @ S | S
S → S # B | B
B → C ~ B | C
C → C ? A | E
A → a
```

2. Write the First() and Follow() of the following Grammar. Draw LL(1) Parsing Table and Parse the string `( ) g c c d c e` showing all steps. If parsing is not possible, point out the reasons. Here, V = {E, U, V, W, X, Y}, T = {c, d, e, (,), g, ε}, and S = {E} **[8]**

```
E → c U V W | X Y E
U → c | ε
V → d U | ε
W → e | ε
X → e | ( Y )
Y → g | ε
```

3. Find True/ False of the following statements: **[5]**

(a) All TDP can use Left Recursive Grammar (LRG).
(b) All TDP can use Non-deterministic Grammar (NDG).
(c) From LRG grammar complete First set can't be found.
(d) From LRG grammar complete Follow set can't be found.
(e) Brute Force Parser is a BUP.

Quiz Question (Total marks: 10)

1. Convert the following Non Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[5]**

```
S → b a c d S | b a c P | b a Q | b a
P → c P | c
Q → a Q | a
```

2. Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[5]**

```
A → A % B | B
B → B & C | C
C → C ^ 0 1 | a b c
```

Answers (exam layout, with comments)

Answer 1 [7]

`WRITE THIS`

The last rule is printed as `C → C ? A | E`. With E there, C can never finish with only terminals, so the grammar makes no strings. Treat it as a typo for ε and write that assumption.

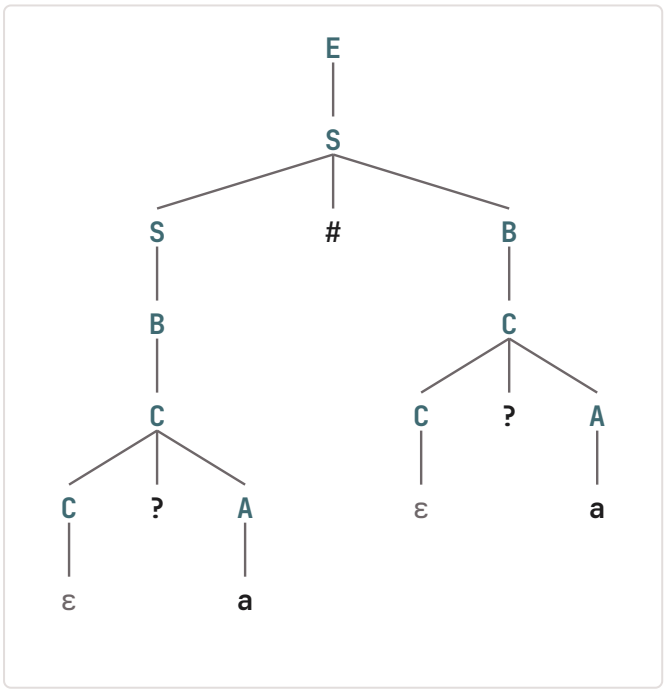
[Assuming the last rule is C → C ? A | ε]

Check every rule for the ambiguity shape X → X op X (the same non-terminal on **both** sides of one operator):

Rule	Recursion
E → E @ S	left only
S → S # B	left only
B → C ~ B	right only
C → C ? A	left only

Each operator is in exactly one rule at one level, so its precedence and associativity are fixed. Example, the string `? a # ? a`: at every step only one rule fits.

LMD: E ⇒ S ⇒ S # B ⇒ B # B ⇒ C # B ⇒ C ? A # B ⇒ ? A # B ⇒ ? a # B ⇒ ? a # C ⇒ ? a # C ? A ⇒ ? a # ? A ⇒ ? a # ? a



Every string has exactly one leftmost derivation parse tree ∴ the grammar is **NOT ambiguous**. No conversion is needed.

Operator	Precedence	Associativity
@	1 (lowest)	left (E → E @ S)
#	2	left (S → S # B)
~	3	right (B → C ~ B)
?	4 (highest)	left (C → C ? A)

> **Lose marks by:** saying "ambiguous" just because there are many operators; not writing the typo assumption; no precedence table.

Answer 2 [8]

WRITE THIS

Numbered rules: 1. E → c U V W 2. E → X Y E 3. U → c 4. U → ε 5. V → d U 6. V → ε 7. W → e 8. W → ε 9. X → e 10. X → (Y) 11. Y → g 12. Y → ε

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
E	{ c, e, (}	{ $ }
U	{ c, ε }	{ d, e, $ }
V	{ d, ε }	{ e, $ }
W	{ e, ε }	{ $ }
X	{ e, (}	{ g, c, e, (}
Y	{ g, ε }	{ c, e, (,) }

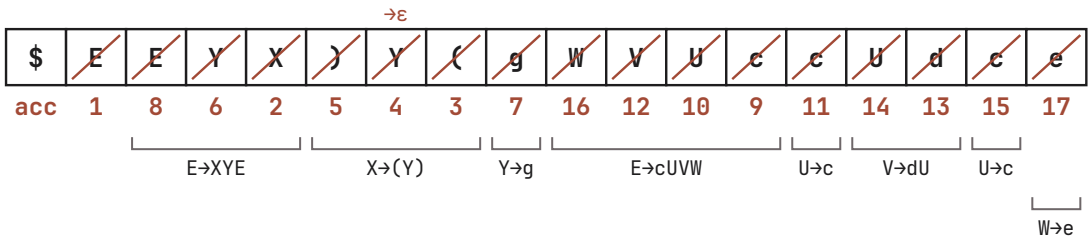
Step 2 · LL(1) parsing table

	c	d	e	(	)	g	$
E	E→cUVW		E→XYE	E→XYE			
U	U→c	U→ε	U→ε				U→ε
V		V→dU	V→ε				V→ε
W			W→e				W→ε
X			X→e	X→(Y)			
Y	Y→ε		Y→ε	Y→ε	Y→ε	Y→g	

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3 · Parse `( ) g c c d c e` (horizontal stack)

Buffer: `( ) g c c d c e $` LA ↑ starts at `(`



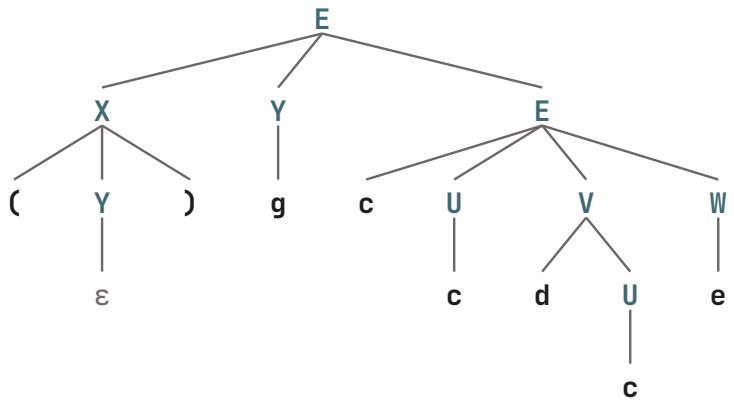
How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop E (LA = (): E→XYE, push E Y X
2. pop X (LA = (): X→(Y), push) Y (
3. match (, pop it, move LA →)
4. pop Y (LA =)): Y→ε, push nothing
5. match), pop it, move LA → g
6. pop Y (LA = g): Y→g, push g
7. match g, pop it, move LA → c
8. pop E (LA = c): E→cUVW, push W V U c
9. match c, pop it, move LA → c
10. pop U (LA = c): U→c, push c
11. match c, pop it, move LA → d
12. pop V (LA = d): V→dU, push U d
13. match d, pop it, move LA → c
14. pop U (LA = c): U→c, push c
15. match c, pop it, move LA → e
16. pop W (LA = e): W→e, push e
17. match e, pop it, move LA → $

acc. Top = $ and LA = $ → accept

Parse tree



Leaves left to right (skip ε): `( ) g c c d c e` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(E): E → c U V W gives { c }. E → X Y E gives { e, (}. Total { c, e, (}.
- FIRST(U): U → c gives { c }. U → ε gives ε. Total { c, ε }.
- FIRST(V): V → d U gives { d }. V → ε gives ε. Total { d, ε }.
- FIRST(W): W → e gives { e }. W → ε gives ε. Total { e, ε }.

- FIRST(X): X → e gives { e }. X → (Y) gives { (}. Total { e, (}.
- FIRST(Y): Y → g gives { g }. Y → ε gives ε. Total { g, ε }.
- FOLLOW(E): start symbol → add $. Total { $ }.
- FOLLOW(U): in E → c U V W: after U is V W → add { d, e }; V W can be ε → add FOLLOW(E). in V → d U: nothing after U → add FOLLOW(V). Total { d, e, $ }.
- FOLLOW(V): in E → c U V W: after V is W → add { e }; W can be ε → add FOLLOW(E). Total { e, $ }.
- FOLLOW(W): in E → c U V W: nothing after W → add FOLLOW(E). Total { $ }.
- FOLLOW(X): in E → X Y E: after X is Y E → add { g, c, e, (}. Total { g, c, e, (}.
- FOLLOW(Y): in E → X Y E: after Y is E → add { c, e, (}. in X → (Y): after Y is) → add {) }. Total { c, e, (,) }.

> **Lose marks by:** forgetting that U, V, W can all be ε (so FOLLOW(U) gets FOLLOW(E)); putting ε rules under FIRST instead of FOLLOW.

Answer 3 [5]

WRITE THIS

#	Answer	Reason (one line)
(a)	**False**	Left recursion makes a top-down parser loop forever (A ⇒ Aα ⇒ Aαα …).
(b)	**False**	Predictive parsers (LL(1)) need a deterministic grammar. Only backtracking parsers cope with NDG, so not ALL TDP.
(c)	**False**	For A → Aα | β, FIRST(A) = FIRST(β) (plus FIRST(α) if β can be ε). Only the naive recursive method loops.
(d)	**False**	FOLLOW comes from where a symbol appears on right sides. Left recursion does not stop this.
(e)	**False**	Brute force = top-down parsing with backtracking.

(c) and (d) are not settled by the slides. This is the textbook view. Always write the reason, so you earn marks either way.

Quiz Answer 1 [5]

WRITE THIS

Round 1: in S, `b a c d S`, `b a c P`, `b a Q`, `b a` all start with `b a`. Take it out once:

```
S → b a S'
S' → c d S | c P | Q | ε
```

Round 2: in S', `c d S`, `c P` all start with `c`. Take it out once:

```
S' → c S'' | Q | ε
S'' → d S | P
```

Round 3: in P, `c P`, `c` all start with `c`. Take it out once:

```
P → c P'
P' → P | ε
```

Round 4: in Q, `a Q`, `a` all start with `a`. Take it out once:

```
Q → a Q'
Q' → Q | ε
```

Final DG:

```
S → b a S'
S' → c S'' | Q | ε
S'' → d S | P
P → c P'
P' → P | ε
Q → a Q'
Q' → Q | ε
```

> **Lose marks by:** stopping after one round (S' still has two alternatives starting with c); forgetting the ε from the bare "ba"; leaving P and Q unfactored.

Quiz Answer 2 [5]

`WRITE THIS`

A → A % B | B: α = `% B`, β = `B`.

```
A → B A'
A' → % B A' | ε
```

B → B & C | C: α = `& C`, β = `C`.

```
B → C B'
B' → & C B' | ε
```

C → C ^ 0 1 | a b c: α = `^ 0 1`, β = `a b c`.

```
C → a b c C'
C' → ^ 0 1 C' | ε
```

Final RRG:

```
A → B A'
A' → % B A' | ε
B → C B'
B' → & C B' | ε
C → a b c C'
C' → ^ 0 1 C' | ε
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

> **Lose marks by:** forgetting ε in the new rules; putting α first (A → αA').